



Actividad | #3 | Backend para el Sistema Web Desarrollo de Sistemas Web II

Ingeniería en Desarrollo de Software

TUTOR: Aarón Iván Salazar Macías

ALUMNO: Alberth Alexis Rodríguez Astorga

FECHA: 20-06-2026

Indice

Introducción.....03

Investigación04

Desarrollo.....05

Conclusión.....09

Referencias.....10

Introducción

En el desarrollo de aplicaciones web modernas, el backend juega un papel en la parte encargada de gestionar la lógica del sistema, el acceso a la base de datos y la comunicación entre el servidor y el cliente.

En esta actividad trabajaremos con la creación de una REST API utilizando Java con Spring Boot, una de las tecnologías más utilizadas en el desarrollo empresarial debido a su eficiencia, seguridad y facilidad de integración.

Una REST API permite realizar operaciones CRUD (crear, leer, actualizar y eliminar) sobre los datos almacenados en una base de datos, utilizando métodos HTTP como GET, POST, PUT y DELETE.

Esto facilita que el frontend pueda comunicarse con el backend de manera estructurada y estandarizada.

Además, el uso de Spring Boot permite agilizar el desarrollo al ofrecer configuraciones automáticas y herramientas que simplifican la creación de servicios web.

En este caso, se implementa un sistema para la gestión de productos, lo que simula el funcionamiento real de una tienda en línea.

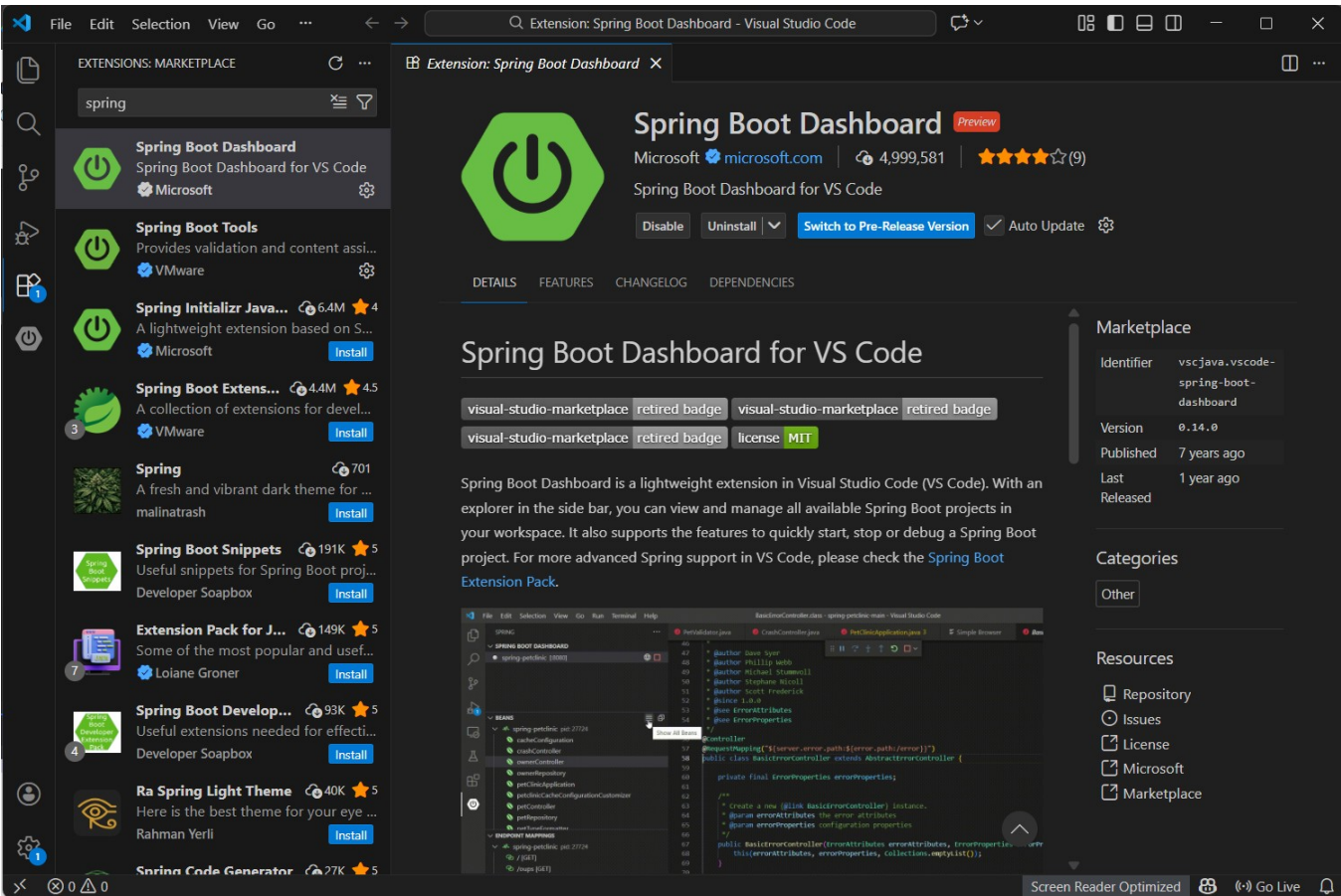
Investigación

Método	Función Principal	Descripción Técnica	Ejemplo de Uso Práctico
GET	Consultar	Obtiene información del servidor de forma segura e idempotente, sin modificar el estado de los datos existentes.	Obtener la lista completa de productos: GET /api/products
POST	Crear	Envía datos en el cuerpo de la petición (body) para crear un nuevo registro en el servidor.	Crear un producto nuevo en el catálogo: POST /api/products
PUT	Actualizar completo	Reemplaza por completo un recurso existente con los nuevos datos enviados. Si falta algún campo, suele sobrescribirse con valores por defecto o nulos.	Actualizar todos los datos de un producto específico: PUT /api/products/1
PATCH	Actualización parcial	Modifica únicamente los campos específicos que se le envían, dejando el resto del recurso intacto.	Cambiar exclusivamente el precio de un artículo: PATCH /api/products/1

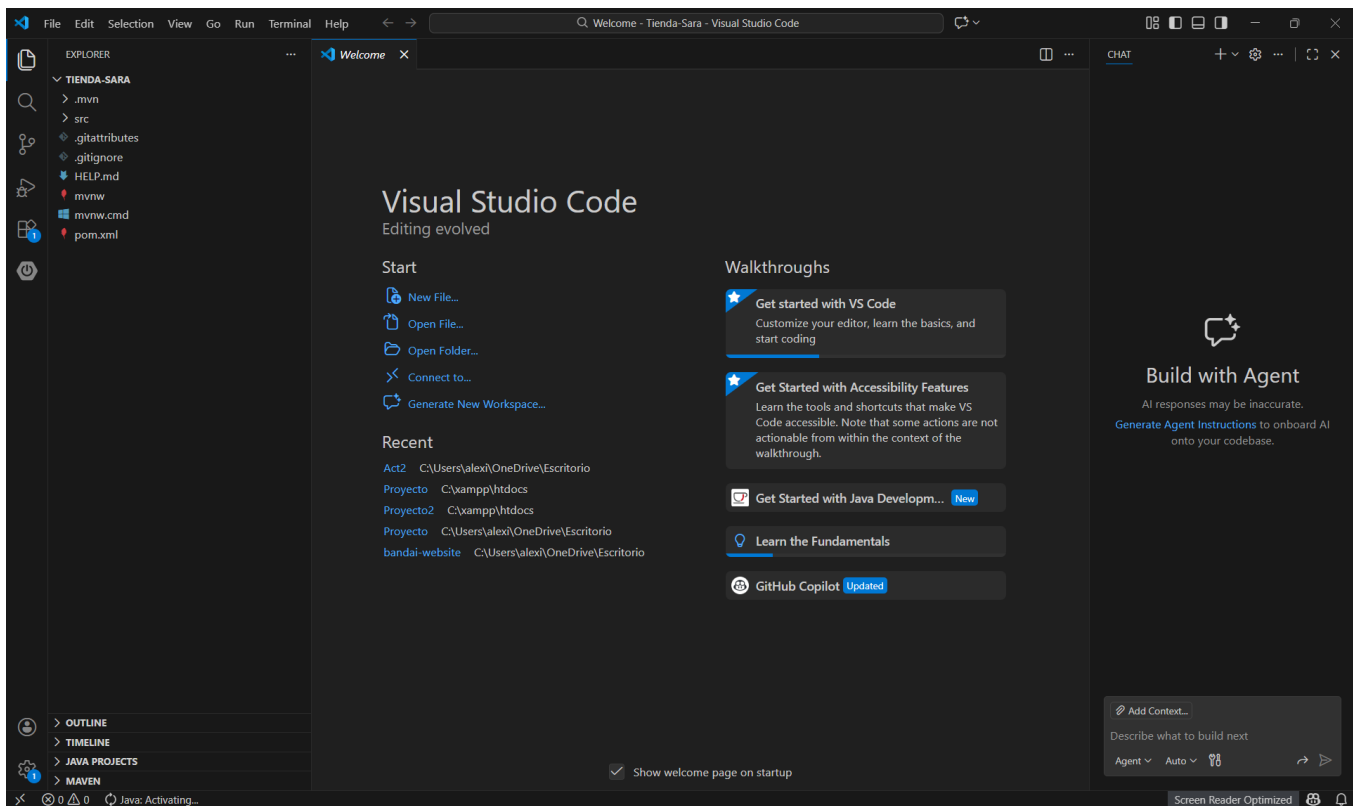
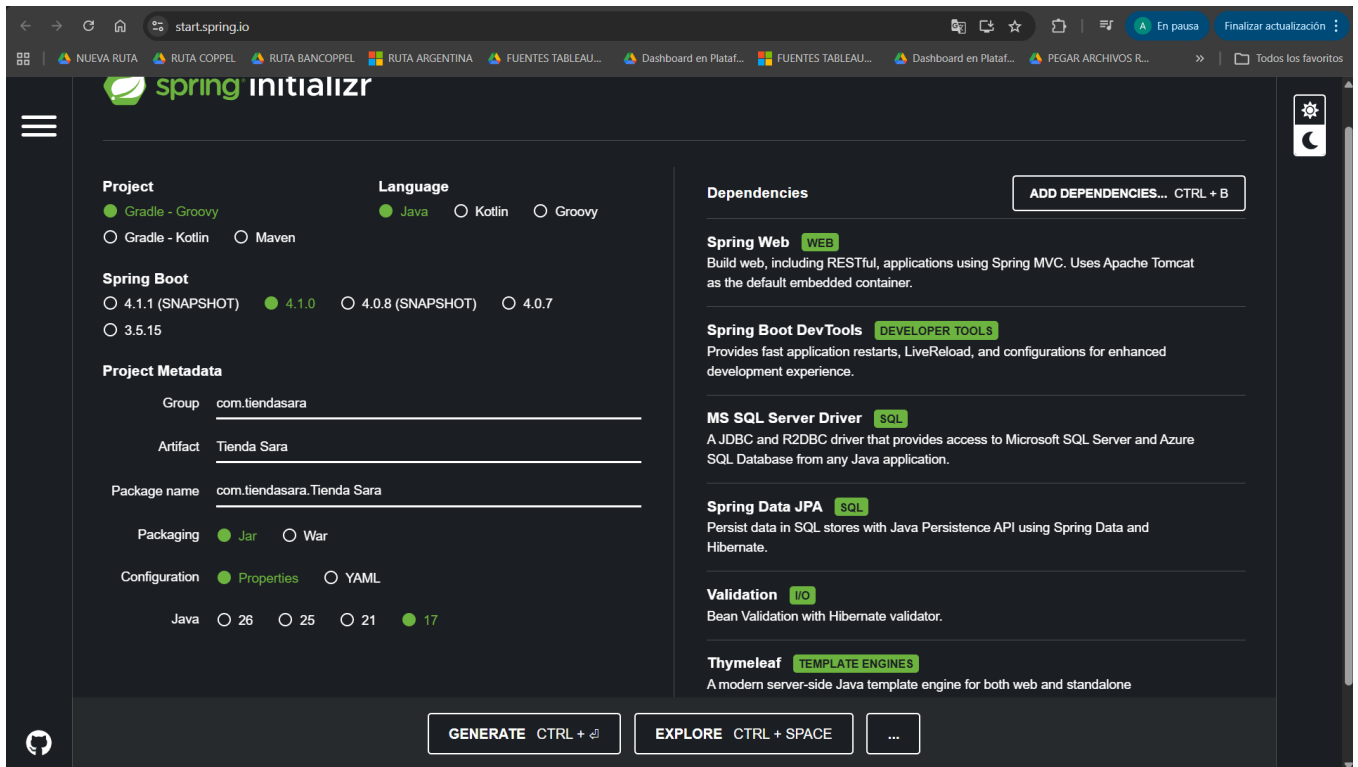
Método	Función Principal	Descripción Técnica	Ejemplo de Uso Práctico
DELETE	Eliminar	Borra de forma definitiva un recurso específico identificado por su ID dentro del servidor.	Eliminar un producto del inventario: DELETE /api/products/1

Desarrollo

Comenzamos instalando las herramientas necesarias para la actividad



Seguimos los pasos para el proyecto:



Paso a paso, hasta llegar al punto de tener la carpeta abierta en el Visual.

Se ejecuta la aplicación en Spring Boot Dashboard:

The screenshot shows the Visual Studio Code interface with the Spring Boot Dashboard on the left. The dashboard lists the application 'Tienda-Sara' under 'APPS' and shows its status as 'Running'. The main editor displays the 'application.properties' file with the following content:

```
src > main > resources > application.properties
1 spring.application.name=Tienda-Sara
2 spring.datasource.url=jdbc:sqlserver://LAPTOP-TR7RCQL2\SQLEXPRESS:1433;trustServerCertificate=
3 spring.datasource.username=sa
4 spring.datasource.password=1234
5 spring.jpa.show-sql=true
6 spring.jpa.hibernate.ddl-auto=update
7
```

The terminal at the bottom shows the command to run the application: `PS C:\Users\alexi\OneDrive\Escritorio\Tienda-Sara> cd 'c:\Users\alexi\OneDrive\Escritorio\Tienda-Sara'; & 'C:\Users\alexi\OneDrive\Escritorio\Tienda-Sara\bin\java.exe' -cp 'c:\Users\alexi\OneDrive\Escritorio\Tienda-Sara\bin\java.exe' 'com.tiendasara.TiendaSaraApplication'`. The output shows the application starting successfully.

The screenshot shows the Visual Studio Code interface with the project structure on the left. The project is named 'Tienda-Sara' and contains the following files and folders:

- mvnw
- .vscode
- src
 - main
 - java
 - com
 - tiendasara
 - controllers
 - ProductController.java
 - models
 - Category.java
 - CategoryDto.java
 - Mark.java
 - MarkDto.java
 - Product.java
 - ProductDto.java
 - services
 - CategoryRepository.java
 - ProductRepository.java
 - resources
 - test
 - target
 - gitattributes
 - gitignore
 - HELP.md
 - mvnw.cmd
 - OUTLINE
 - TIMELINE
 - LOGICAL STRUCTURE
 - JAVA PROJECTS
 - MAVEN

The main editor displays the 'index.html' file with the following content:

```
src > main > resources > templates > products > index.html
1 Generate code (Ctrl+I), or select a language (Ctrl+K M). Start typing to dismiss or don't show this again.
```

The terminal at the bottom shows the command to run the application: `PS C:\Users\alexi\OneDrive\Escritorio\Tienda-Sara> cd 'c:\Users\alexi\OneDrive\Escritorio\Tienda-Sara'; & 'C:\Users\alexi\OneDrive\Escritorio\Tienda-Sara\bin\java.exe' -cp 'c:\Users\alexi\OneDrive\Escritorio\Tienda-Sara\bin\java.exe' 'com.tiendasara.TiendaSaraApplication'`. The output shows the application starting successfully.

Creamos todas las carpetas y archivos dentro.

Comienzo a pegar los códigos proporcionados para la actividad en sus respectivos archivos.

The screenshot shows the Visual Studio Code interface with the `launch.json` file open in the editor. The file contains two configurations for running the application. The first configuration is for a standard launch, and the second is for a debug launch. The terminal shows the output of the `spring-boot:run` command, indicating a successful build and the start of the application.

```
.vscode > launch.json > ...
3  "configurations": [
4    {
5      "mainClass": "com.tiendasara.TiendaSaraApplication",
6      "projectName": "Tienda-Sara",
7      "args": "",
8      "envFile": "${workspaceFolder}/.env"
9    },
10   {
11     "type": "java",
12     "name": "Spring Boot-TiendaSaraApplication<Tienda-Sara>",
13     "request": "launch",
14     "cwd": "${workspaceFolder}",
15     "mainClass": "com.tiendasara.Tienda.Sara.TiendaSaraApplication",
16     "projectName": "Tienda-Sara",
17     "args": "",
18     "envFile": "${workspaceFolder}/.env"
19   }
20 ]
21
22
23
24
25 }
```

```
PS C:\Users\alexi\OneDrive\Escritorio\Tienda-Sara> .\mvnw.cmd spring-boot:run
at org.springframework.boot.jdbc.autoconfigure.DataSourceConfiguration$Hikari.dataSource(Data
SourceConfiguration.java:128) ~[spring-boot-jdbc-4.1.0.jar:4.1.0]
at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccesso
r.java:104) ~[na:na]
at java.base/java.lang.reflect.Method.invoke(Method.java:565) ~[na:na]
at org.springframework.beans.factory.support.SimpleInstantiationStrategy.lambda$instantiate$0
(SimpleInstantiationStrategy.java:155) ~[spring-beans-7.0.8.jar:7.0.8]
... 40 common frames omitted

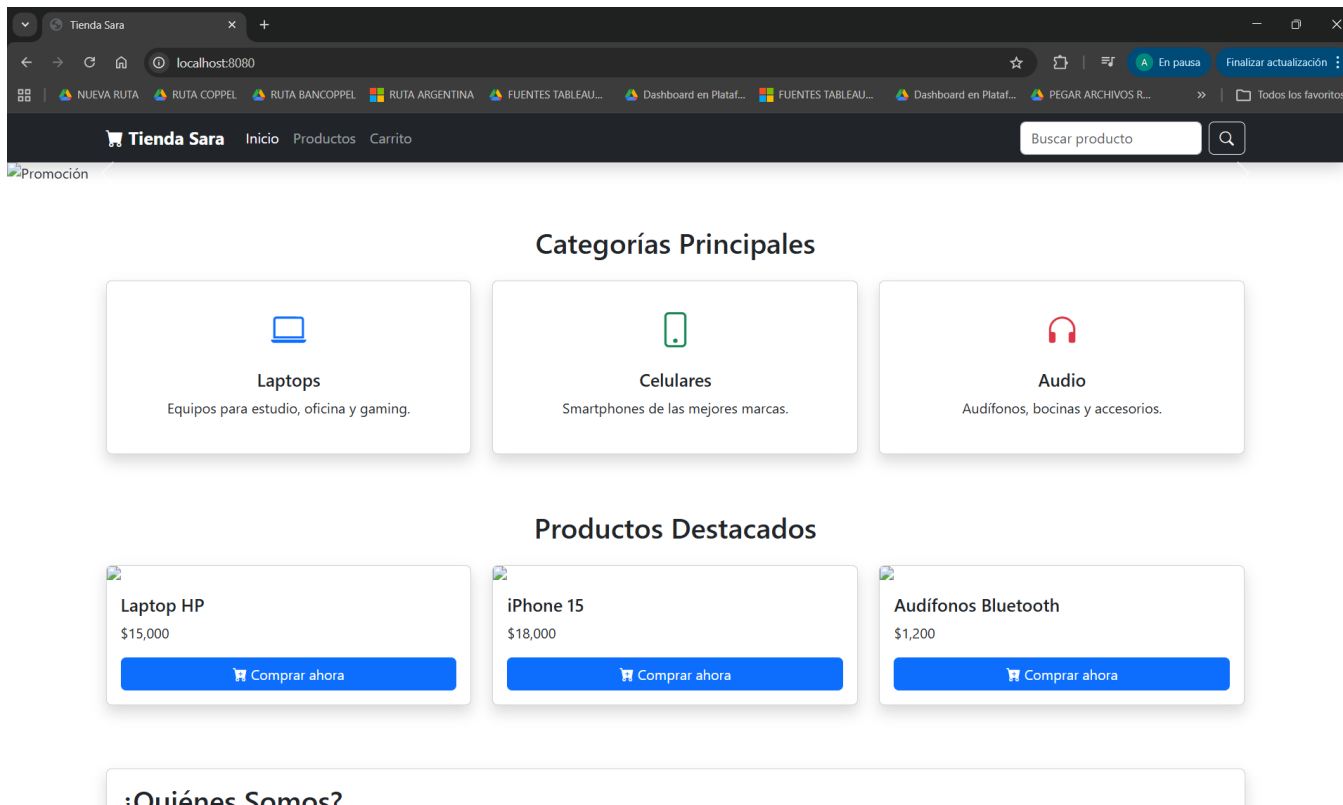
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.229 s
[INFO] Finished at: 2026-06-22T00:08:52-07:00
[INFO] -----
PS C:\Users\alexi\OneDrive\Escritorio\Tienda-Sara>
```

The screenshot shows the Visual Studio Code interface with the `application.properties` file open in the editor. The file contains the configuration for the application, including the database URL, username, password, and other settings. The terminal shows the output of the `spring-boot:run` command, indicating a successful build and the start of the application.

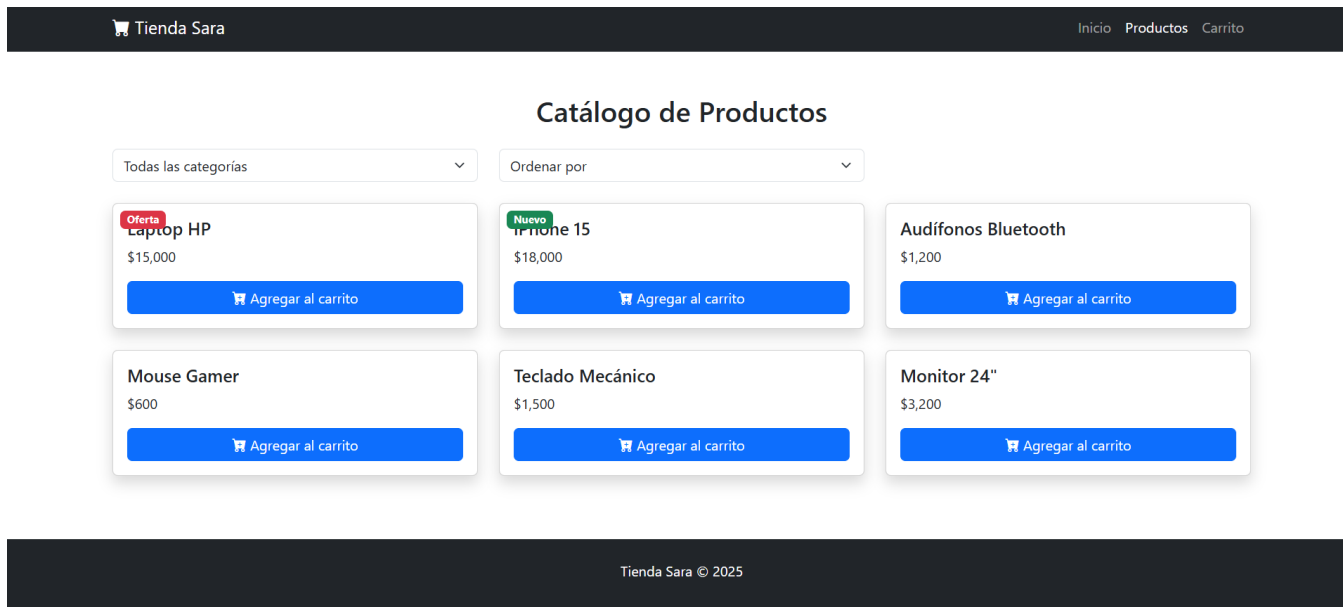
```
src > main > resources > application.properties
1  spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=Prueba;encrypt=true;trustSe
2  spring.datasource.username=sa
3  spring.datasource.password=1234
4  spring.jpa.show-sql=true
5  spring.jpa.hibernate.ddl-auto=validate
```

```
y configure spring.jpa.open-in-view to disable this warning
2026-06-22T01:16:53.882-07:00 INFO 9412 --- [ restartedMain] o.s.b.w.a.WelcomePageHandlerMapping : Adding we
lcome page: class path resource [static/index.html]
2026-06-22T01:16:54.025-07:00 INFO 9412 --- [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat st
arted on port 8080 (http) with context path '/'
2026-06-22T01:16:54.028-07:00 INFO 9412 --- [ restartedMain] com.tiendasara.TiendaSaraApplication : Started T
iendaSaraApplication in 1.598 seconds (process running for 100.334)
2026-06-22T01:16:54.032-07:00 INFO 9412 --- [ restartedMain] .conditionEvaluationDeltaLoggingListener : Condition
evaluation unchanged
2026-06-22T01:17:20.624-07:00 INFO 9412 --- [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializ
ing Spring DispatcherServlet 'dispatcherServlet'
2026-06-22T01:17:20.624-07:00 INFO 9412 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Initializ
ing Servlet 'dispatcherServlet'
2026-06-22T01:17:20.624-07:00 INFO 9412 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed
initialization in 0 ms
```

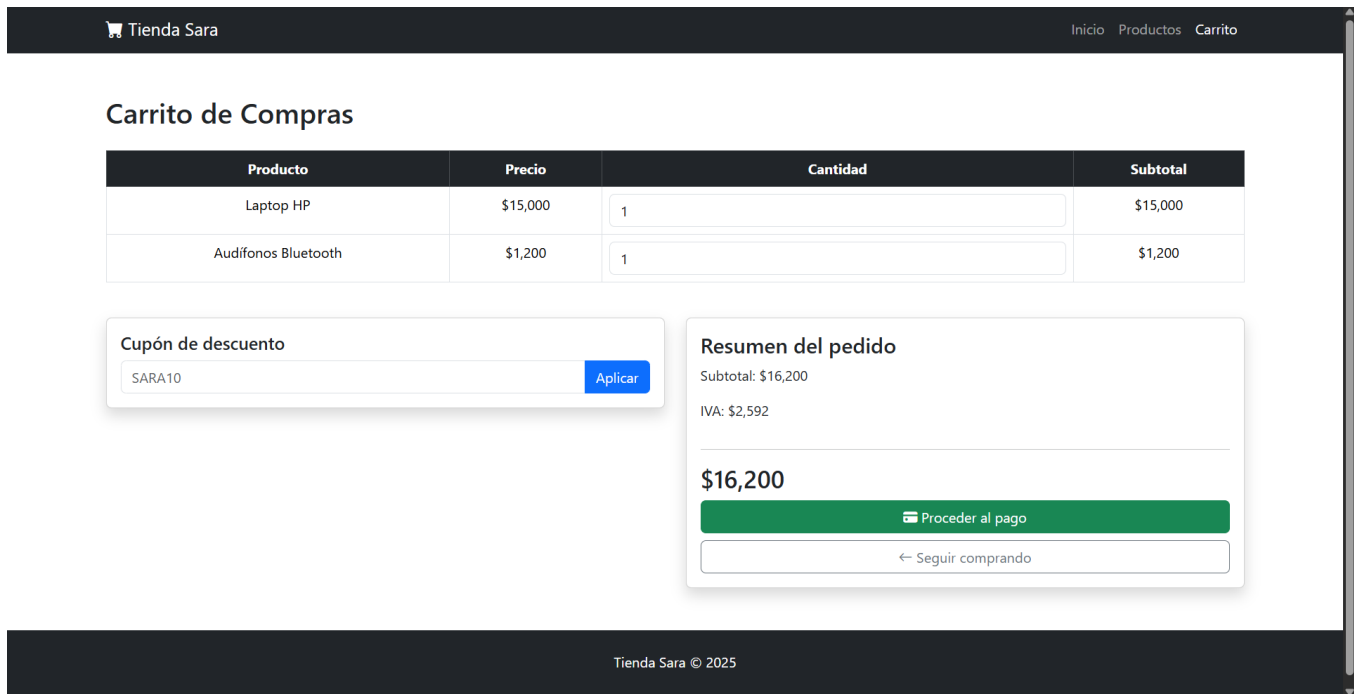
Parece que ya quedo.



La pagina ya aparece en el <http://localhost:8080/>



Ya muestra la información.



Conclusión

En esta actividad se logró integrar una aplicación desarrollada en Spring Boot con una base de datos SQL Server, identificando y resolviendo problemas comunes de conexión entre ambos sistemas.

Durante el proceso se evidenció la importancia de una correcta configuración del entorno, especialmente en aspectos como el estado de los servicios de SQL Server, la habilitación del protocolo TCP/IP y la correcta configuración del servidor e instancia.

El error principal presentado estuvo relacionado con la imposibilidad de establecer conexión con la instancia SQLEXPRESS, lo que se debió a servicios detenidos o configuración de red insuficiente.

A través del análisis de los mensajes de error y la revisión de la configuración del sistema, se comprendió cómo interpretar fallos en aplicaciones Java con Hibernate y cómo estos se relacionan directamente con la infraestructura de la base de datos.

Esta actividad demostró que construir un sistema no solo requiere escribir código limpio, sino también entender cómo interactúa el servidor de aplicaciones con los puertos de red del sistema operativo y con las restricciones del motor de base de datos relacional.

Concepto Clave	Aplicación Práctica en la Actividad	Impacto en el Aprendizaje
Conexión Java con Bases de Datos	Enlace del ecosistema Java directamente con SQL Server mediante el driver oficial de Microsoft, permitiendo que la aplicación web interactúe directamente con los esquemas relacionales mapeados.	Se consolidó la comprensión de cómo los datos viajan desde el backend hacia una base de datos persistente en un entorno de producción local.
Uso de Spring Boot y JPA (Hibernate)	Implementación de repositorios de Spring Data JPA (JpaRepository) y uso de estrategias ddl-auto (update/validate) para la creación automática de tablas relacionales (productos, categorías, marcas).	Se eliminó la necesidad de escribir SQL manual para el CRUD básico, asimilando las ventajas de la inyección por constructor y la abstracción orientada a objetos.
Diagnóstico de Errores en SQL Server	Identificación y resolución de excepciones críticas en tiempo de ejecución, como el fallo por discrepancias de nombres de columnas en consultas generadas (Invalid column name 'id').	Se mejoró la capacidad de leer stack traces de error de Hibernate para depurar rápidamente incoherencias entre los atributos de las entidades Java y las columnas reales en las tablas.
Configuración de Servicios y Red Local	Manejo de conflictos de red en el host local, liberación manual de puertos mediante comandos de consola (taskkill) y reasignación de puertos del servidor en application.properties (server.port=8080).	Se comprendió la importancia de gestionar correctamente el ciclo de vida de los procesos de red y los servicios de fondo (como Apache Tomcat), evitando bloqueos y colisiones entre ejecuciones concurrentes.

Referencias

Código de GitHub:

<https://github.com/AlexisRodriguez1988/Desarrollo-de-Sistemas-Web-II/tree/main>

Microsoft. (2025). *Microsoft JDBC Driver for SQL Server* (Versión 13.4) [Software].

<https://learn.microsoft.com/sql/connect/jdbc/microsoft-jdbc-driver-for-sql-server>